# Deck 06 · Motion planning: optimization and sampling

**4.3 From a goal configuration to a motion**

**4.4 Find routes, refine them, and check them**

Özgür S. Öğüz · Bilkent University · Fall 2026

**Known:** forward kinematics, local optimization, constrained inverse kinematics.

**New task:** connect the current posture to an allowed goal.

# Waypoint + reference motion + controller

**Goal:** reach the desired tip position at a chosen future time.

**1 · Waypoint**
IK chooses an allowed goal posture for the target.

→

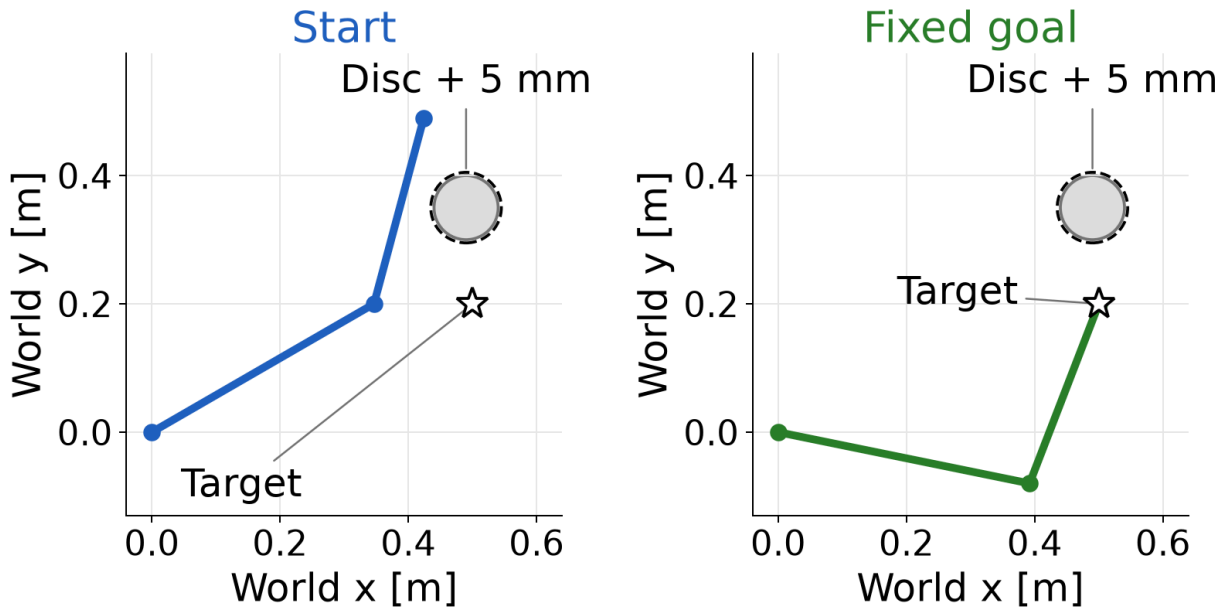
**2 · Reference motion**
Plan and check a route; choose when to traverse it.

→

**3 · Controller**
Use the reference and measurements to send robot commands.

**Three horizons:** future goal → motion along the route → short-term corrections.

# An allowed goal still needs a connecting motion



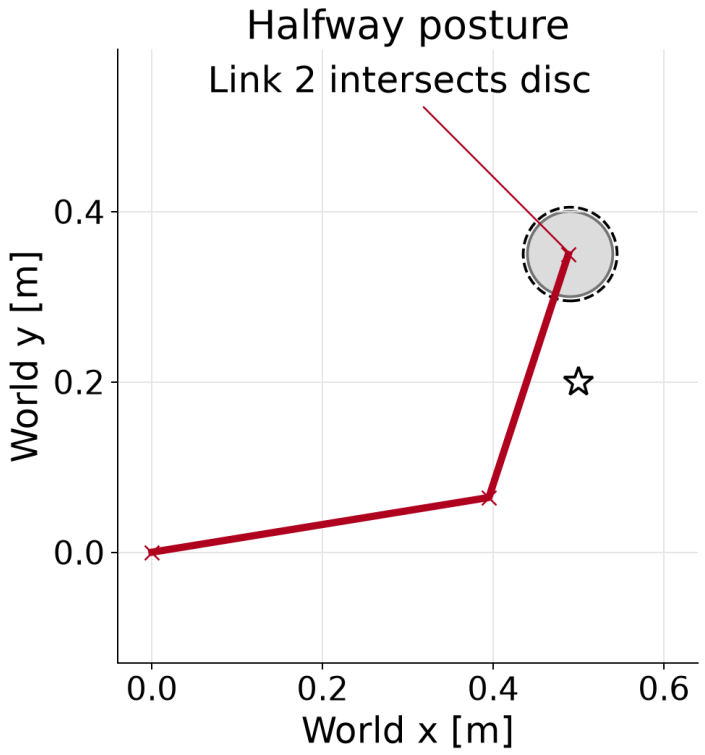

**Model:** 0.4 m and 0.3 m whole links; 5 mm clearance from the disc.

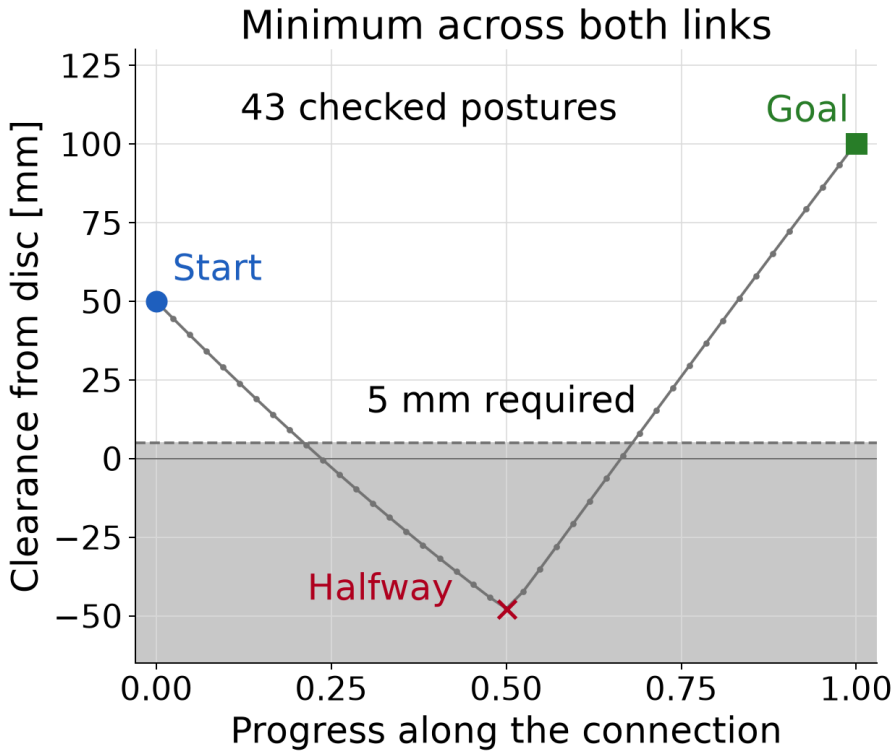
**Joint range:** –180° to 180° each; no wrapping or 60° elbow limit.

**Endpoints:** $(30^\circ, 45^\circ)$ to $(-11.517^\circ, 80.406^\circ)$; tip goal $(0.5, 0.2)$ m.

**Predict:** move both angles straight to their goals. Does the arm stay clear?

# The direct connection crosses the obstacle

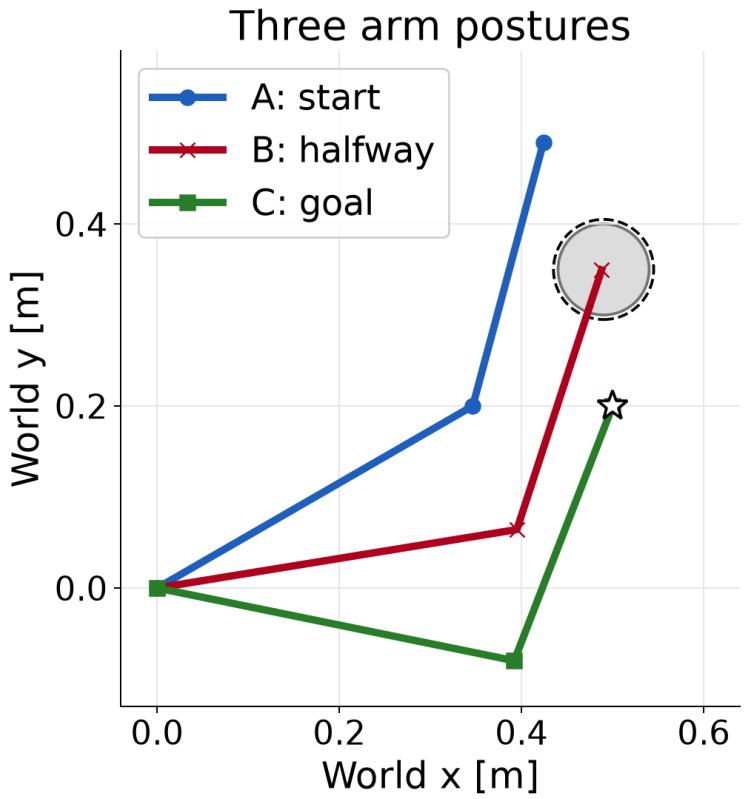


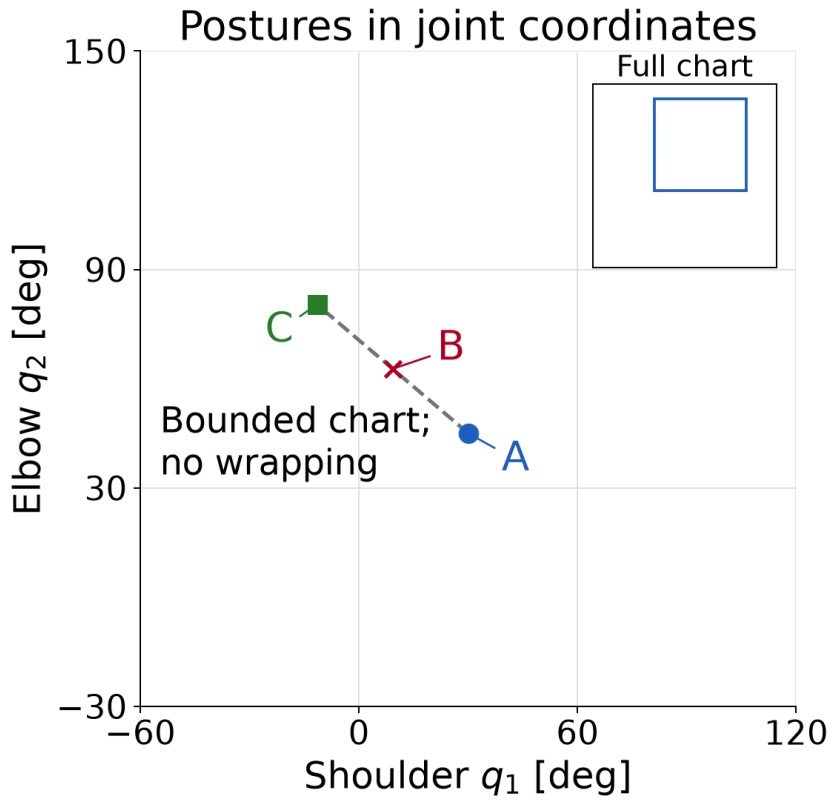




**Halfway posture:** second link penetrates the disc by 47.7 mm.

**Clear endpoints do not establish a clear connection.**

# One arm posture becomes one point

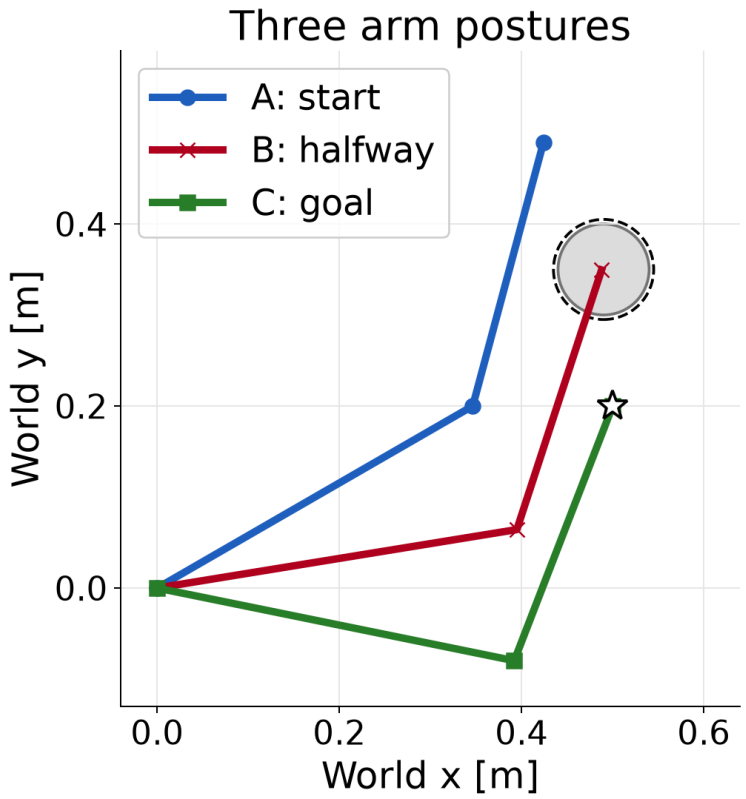


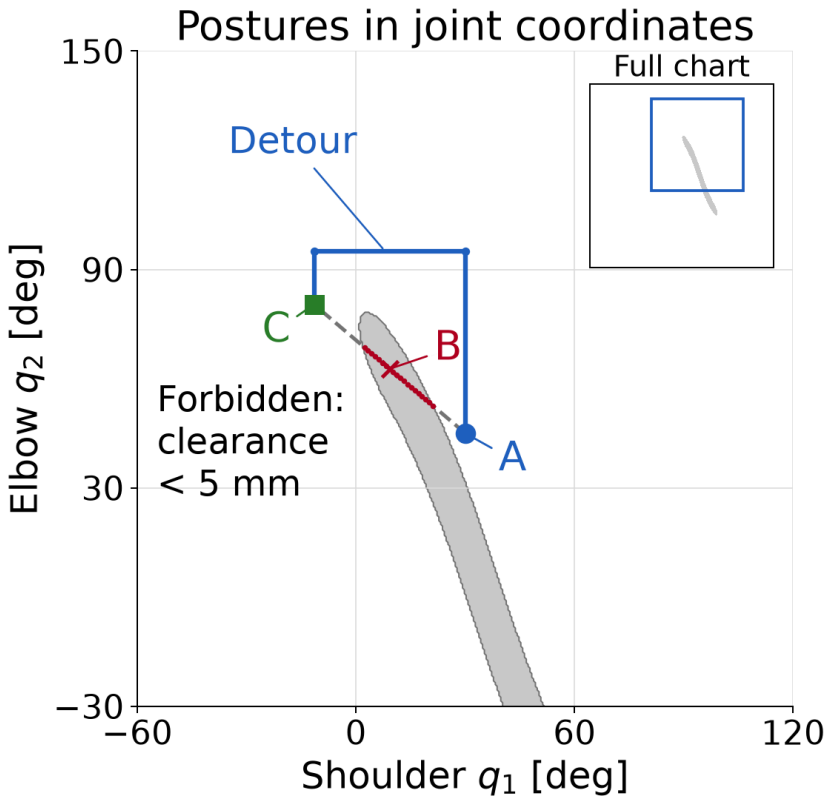




**Horizontal coordinate:** shoulder angle. **Vertical coordinate:** elbow angle.

The pair of angles specifies the whole arm posture.

# Forbidden postures form regions




$$\mathcal{Q} = [-\pi, \pi]^2, \qquad \mathcal{Q}_{\text{free}} \subseteq \mathcal{Q}$$

**Allowed:** both links at least 5 mm clear. **Shaded:** violations. Axes use degrees.

# A path specifies the route

**Given:** robot and obstacles, coordinate bounds, clearance rule, fixed start and goal.

**Find a continuous path through allowed configurations:**

$$\mathbf{q}_{\text{path}}(s) \in \mathcal{Q}_{\text{free}}, \qquad 0 \le s \le 1$$

$$\mathbf{q}_{\text{path}}(0) = \mathbf{q}_{\text{start}}, \qquad \mathbf{q}_{\text{path}}(1) = \mathbf{q}_{\text{goal}}$$

$s$: progress along the path; no time units.

**A waypoint list needs a rule for connecting adjacent waypoints.**

# Timing specifies when each posture is reached

**Same route, different progress schedules:**

$$\mathbf{q}(t) = \mathbf{q}_{\text{path}}(s(t))$$

$$s(0) = 0, \qquad s(t_{\text{f}}) = 1$$

$t$: time in seconds. $t_{\text{f}}$: duration. $s(t)$: continuous, nondecreasing progress.

| Geometric path | Timed trajectory |
|---|---|
| Which configurations, in which order | Which configuration at each time |
| Clearance along the route | Velocity and acceleration also matter |

# A connection needs checks between its endpoints

**Configuration check:** whole-link clearance at one posture.

**Edge check:** interpolate joint angles and test intermediate postures.

$$\mathbf{q}_{\text{edge}}(s) = (1 - s)\mathbf{q}_a + s\mathbf{q}_b, \qquad 0 \le s \le 1$$

| Rule for this scene | Direct start-to-goal edge |
|---|---|
| At most 1° per joint between checks | 42 subintervals; 43 postures (indices 0–42) |
| At least 5 mm clearance at every check | First failure at index 9; reject the edge |

**One failed intermediate posture rejects the edge.**

Passing checks establishes clearance only at the checked postures.

rrt_two_link.py --edge ▶ check the edge live

# Choose the free knots together

$$\mathbf{z} = (\mathbf{q}_1, \ldots, \mathbf{q}_T) \in \mathbb{R}^{nT}$$

- Current posture $\mathbf{q}_0$: fixed
- Remaining $T$ postures: chosen together by trajectory optimization
- $n = 2, T = 20$: **40 decision coordinates**, 21 postures including the start

$t$ is a **discrete configuration index**; RAI calls the $T$ free ones **time slices**.

$$\Delta t = \frac{1}{T}, \qquad \text{normalized time at knot } t = \frac{t}{T}$$

**Predict:** does one optimizer step move only the colliding knots?

Existing browser example: soft whole-link penalty, no 5 mm margin ▶ run it live

# Joint-angle differences give angular velocity and acceleration

**Joint angular velocity over one interval:**

$$\mathbf{v}_t = \frac{\mathbf{q}_t - \mathbf{q}_{t-1}}{\Delta t}$$

**Joint angular acceleration from consecutive velocities:**

$$\mathbf{a}_t = \frac{\mathbf{v}_t - \mathbf{v}_{t-1}}{\Delta t} = \frac{\mathbf{q}_{t-2} - 2\mathbf{q}_{t-1} + \mathbf{q}_t}{\Delta t^2}$$

Joint angular velocity uses **two knots**. Angular acceleration uses **three**.

These normalized-time differences supply the trajectory cost; physical duration comes later.

# Specify the start and scale the smoothness cost

**Fixed prefix for an initially resting robot:**

$$\mathbf{q}_{-1} := \mathbf{q}_0, \qquad \mathbf{a}_1 = \frac{\mathbf{q}_1 - \mathbf{q}_0}{\Delta t^2}$$

**Smoothness cost:** approximate integrated squared angular acceleration.

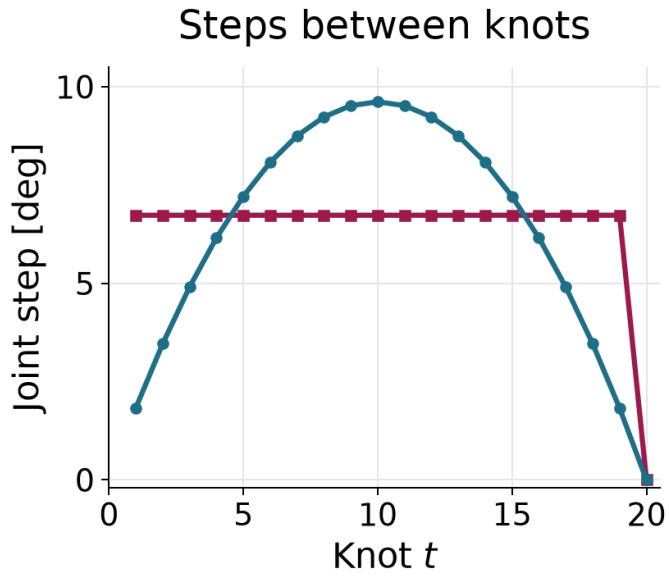
$$f(\mathbf{z}) = \sum_{t=1}^{T} \Delta t \, \|\mathbf{a}_t\|^2 = \sum_{t=1}^{T} \|\mathbf{r}_t\|^2$$

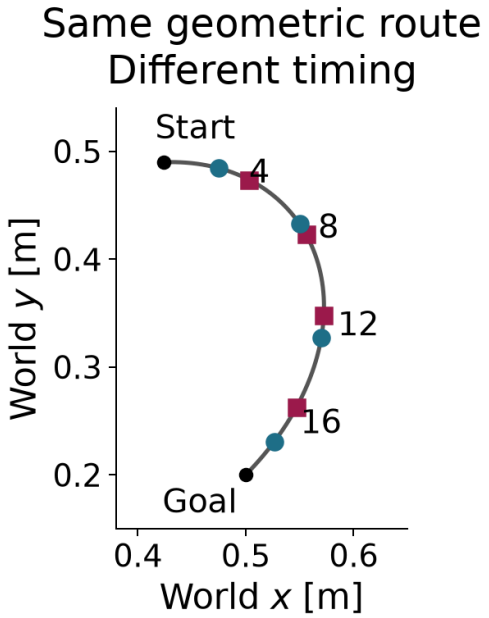
$$\mathbf{r}_t = \sqrt{\Delta t}\, \mathbf{a}_t \quad (\text{acceleration}), \qquad \mathbf{r}_t = \sqrt{\Delta t}\, \mathbf{v}_t \quad (\text{velocity})$$

**This cost ranks motions; obstacle clearance is a separate requirement.**

The first step is penalized; it is **not forced to zero**.

# Velocity cost against acceleration cost




Velocity cost  Acceleration cost

**Same start, negative-elbow goal and final rest.**

| Cost | Joint step sizes |
|---|---|
| Velocity: first differences | 19 steps of 6.73°, then zero |
| Acceleration: second differences | 1.83° → 9.62° → zero |

Same geometric route; tips up to **3.8 cm apart at matching knots**.

Live comparison: **disc removed**.

komo_two_link.py --k1 · Trajectory tab ▶ velocity cost ▶ acceleration cost

# Costs choose motion; constraints state requirements

**Example: fixed goal configuration and terminal rest**

$$\mathbf{h}(\mathbf{z}) = \begin{pmatrix} \mathbf{q}_T - \mathbf{q}_{\text{goal}} \\ (\mathbf{q}_T - \mathbf{q}_{T-1})/\Delta t \end{pmatrix} = \mathbf{0}$$

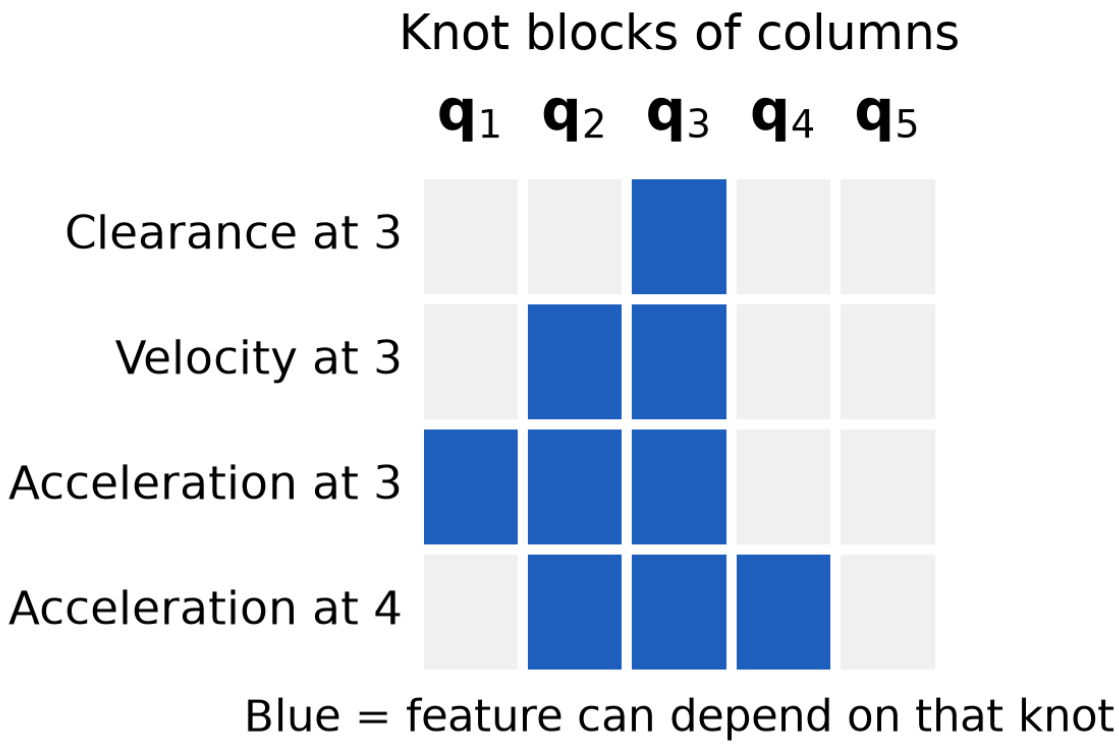
**Both complete links at every free time slice:**

$$g_{t,\ell} = r_o + d_{\text{req}} - \text{dist}(\mathbf{c}, \mathcal{B}_\ell(\mathbf{q}_t)) \leq 0$$

$\mathcal{B}_\ell$: link segment; $\mathbf{c}$: disc center; $r_o$: disc radius; $d_{\text{req}} = 5\,\text{mm}$.

**Twenty slices × two links = forty clearance rows.**

# Each feature depends on nearby knots




**Columns:** joint coordinates, grouped by knot.

**Rows:** motion or task feature groups.

| Feature | Knots used |
|---|---|
| Clearance | One |
| Velocity | Two neighbors |
| Acceleration | Three neighbors |

**Sparse Jacobian; block-banded normal matrix.**

# Neighboring-knot features define the KOMO program

$$\min_{\mathbf{q}_{1:T}} \sum_{t=1}^{T} \left\| \mathbf{r}_t(\mathbf{q}_{t-k:t}) \right\|^2$$

$$\mathbf{g}_t(\mathbf{q}_{t-k:t}) \leq \mathbf{0}, \qquad \mathbf{h}_t(\mathbf{q}_{t-k:t}) = \mathbf{0}$$

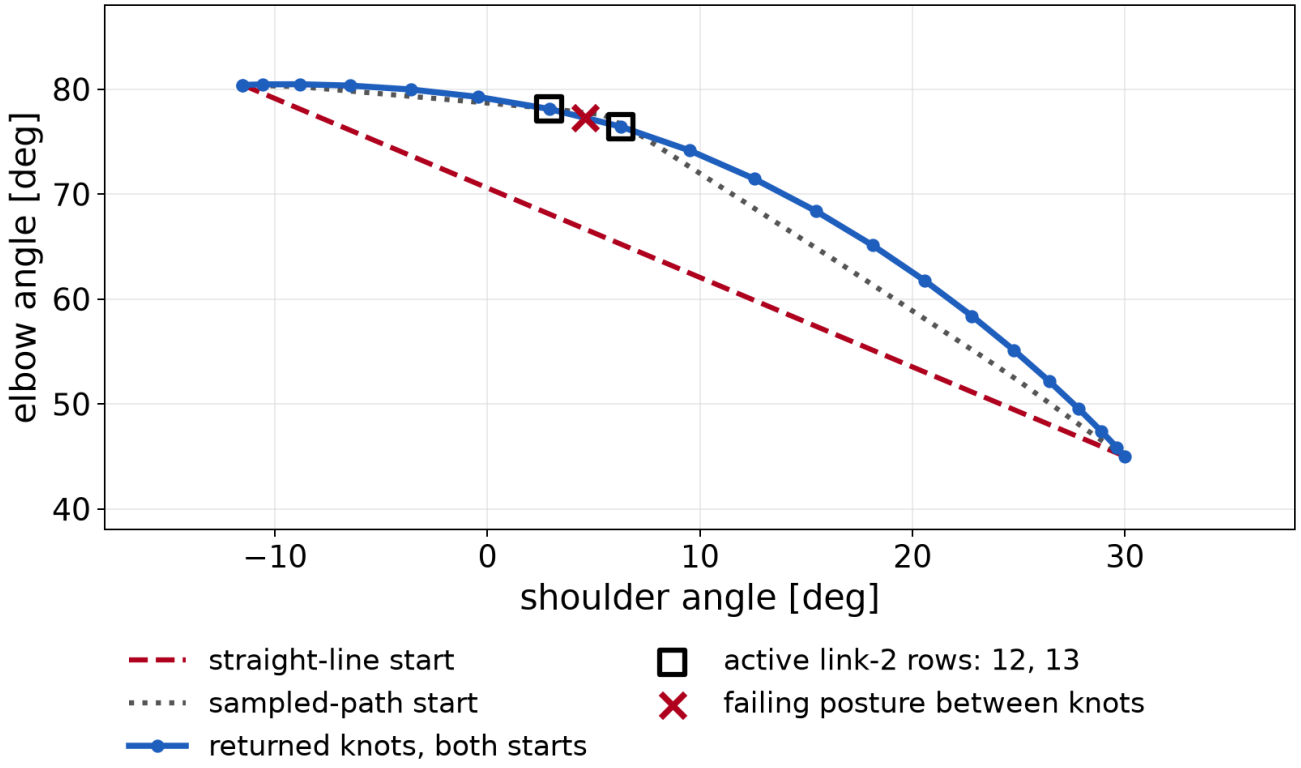
$k$: largest difference order; here $k = 2$.

**k-order Markov Optimization (KOMO)** describes the structured formulation.

Deck 05's **augmented Lagrangian**, with damped Gauss–Newton inside, adjusts the knot sequence.

**Local structure helps computation. It does not imply a global optimum.**

# Worked example: KOMO checks both complete links




**Fixed goal; 5 mm whole-link rows at 20 knots.** Cost $16.14$; active link-2 rows at knots 12–13.

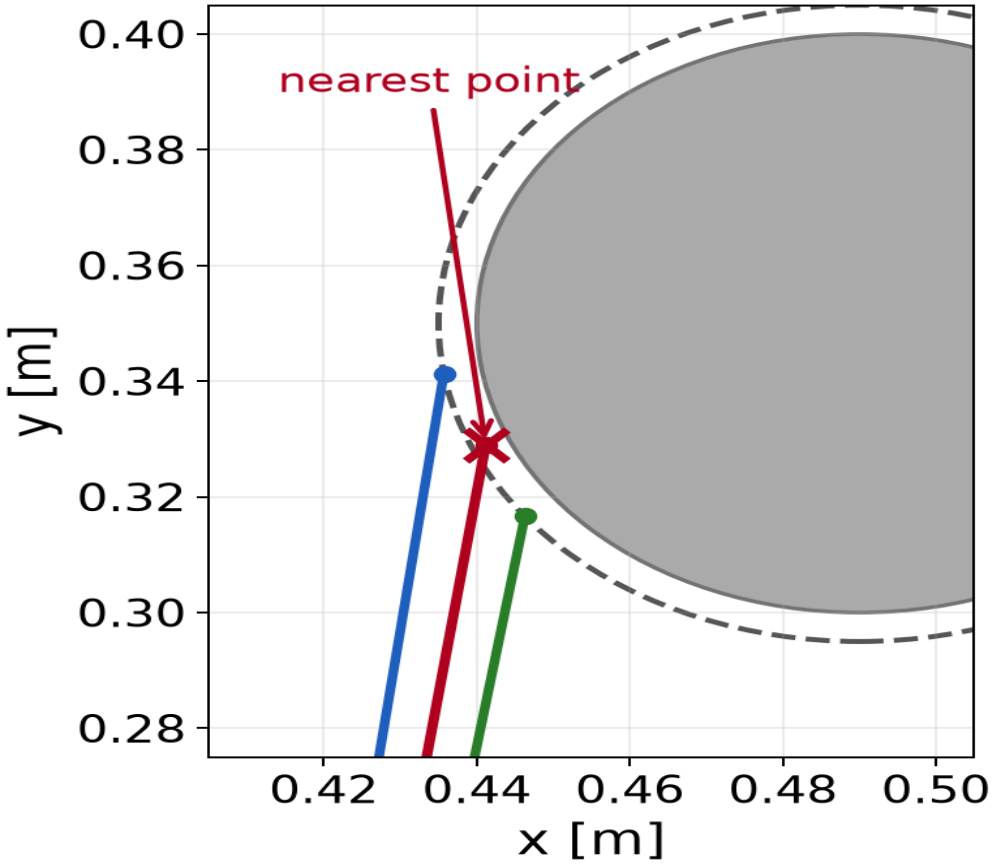
# Two starts reach the same knot sequence here

| | Straight-line start | Sampled-path start |
|---|---|---|
| Initial cost | $18.14$ | $28.78$ |
| Clearance rows initially | Violated | All pass |
| Final cost | $16.1382$ | $16.1382$ |
| Active rows | Link 2, knots 12–13 | Same |
| Solver stop | KKT thresholds, 5 outer steps | KKT thresholds, 5 outer steps |

**The sampled start changes the initial path, but not this answer.**

**Both outputs still need a between-knot check.**

# Shape-clear knots can hide an invalid connection




**Link 2, knots 12 → 13**

Blue and green: knot postures. Red: halfway posture.

Dashed: 5 mm ring. Gray: physical disc.

Both knot rows meet the **5 mm** requirement.

Halfway between them: **1.8 mm inside the clearance ring**.

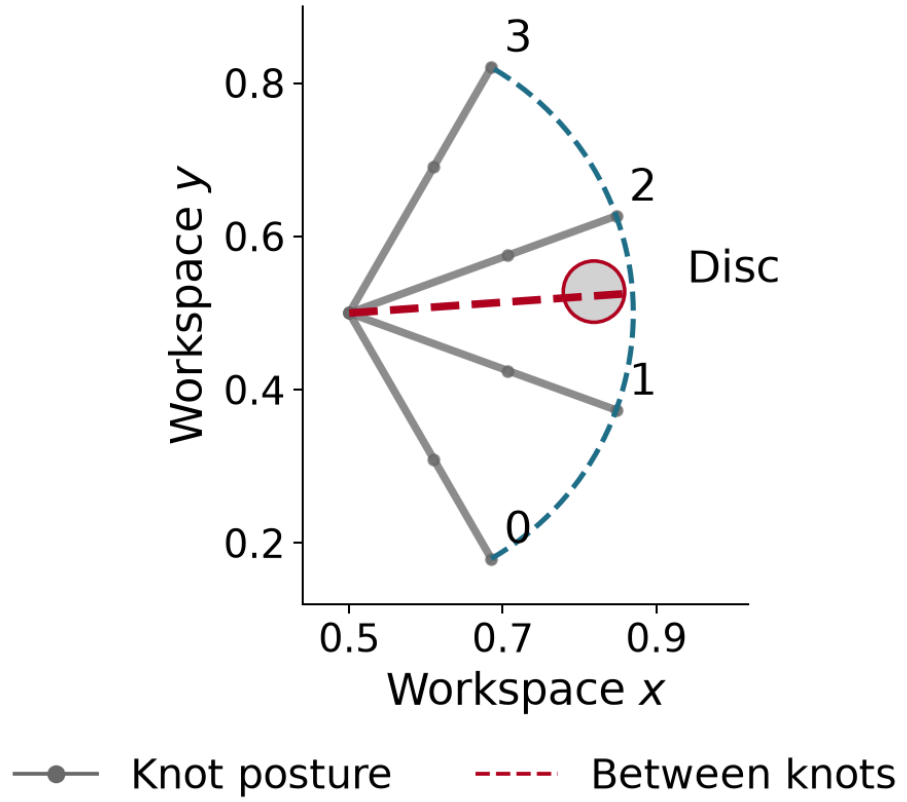
The link stays outside the physical disc.

# Live demo: clear knots can hide a collision

- Opens on the **Trajectory** tab with **Between-knot collision** loaded: four arm poses, all clear; obstacle cost **0**
- **Predict:** is the motion between those poses clear too?
- Interpolate the joints: the **red dashed arm collides**
  - knot clearance: **+0.0428**
  - sampled-motion clearance: **–0.0344**
- Choose **Add midpoint knots**: same joint path, seven knots, now a **positive obstacle cost**

**Soft whole-link penalty; separate arm, normalized units.**

More knots reveal the collision; they do not repair the path.




modules/opt-explorer · Trajectory tab · Between-knot collision ▶ run it live

# Solver convergence leaves motion checks to do

- **Geometry:** both complete links must meet the 5 mm rule at checked postures
- **Time:** the optimized knots pass, but a connection misses the ring by **1.8 mm**
- **Initialization:** two different starts reach the same answer in this scene
- **Changed path:** repeat whole-link and edge checks after each modification

**KKT residuals assess the written finite-knot program.**

# Break

We can optimize a knot sequence and inspect what its constraints omit.

**Next:** construct candidate routes by testing sampled connections.

# Search using geometry checks as needed

| Operation | Choice in this scene |
|---|---|
| Draw a candidate posture | Uniform joint-box sample; the tree planner also samples the goal |
| Compare postures | Euclidean joint-angle distance, in radians |
| Propose a connection | Straight joint-space segment, no wrapping |
| Accept or reject | Whole-link 5 mm clearance; at most 1° per joint |

**We query the geometry; we do not build its full forbidden region.**

# Store checked connections in a reusable graph

1. Draw configurations and retain the valid ones.
2. Choose nearby pairs and test their connecting edges.
3. Store accepted edges with their joint-space lengths.
4. Attach the start and goal, then search the graph.

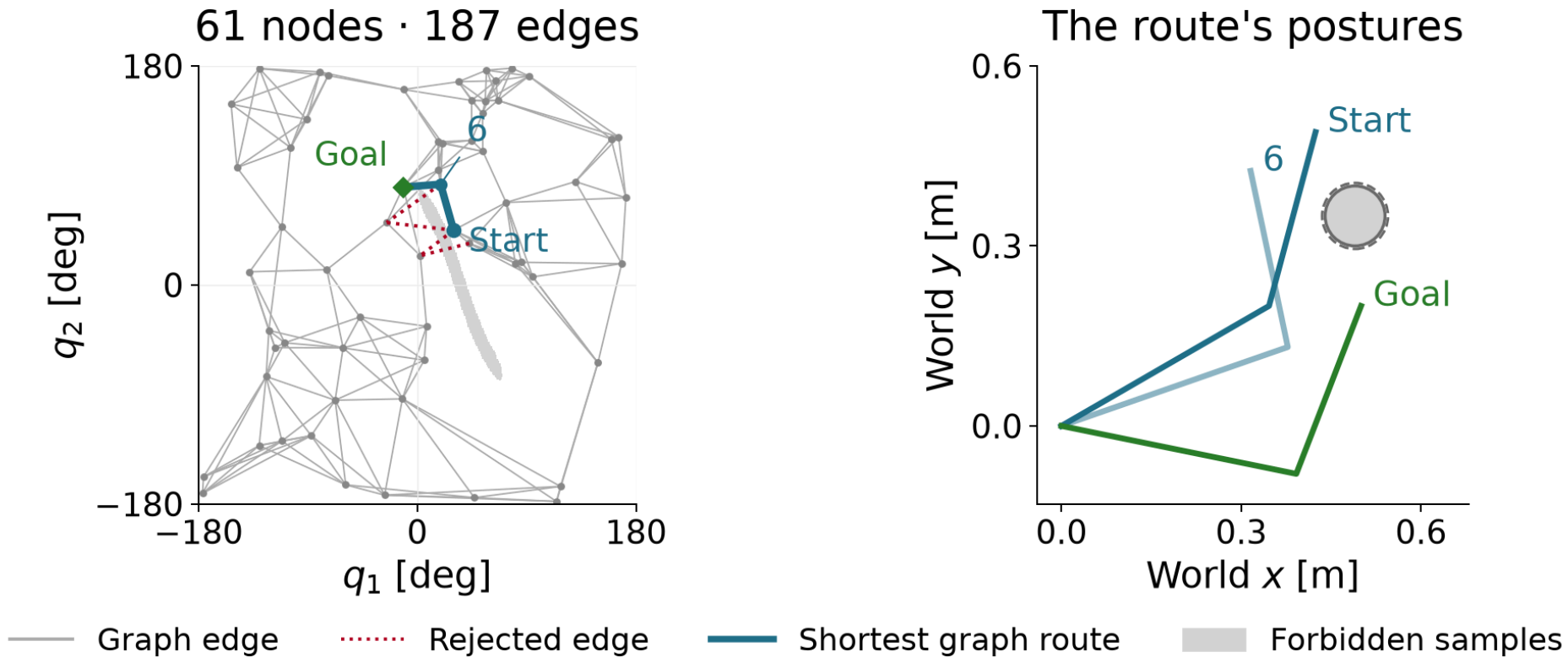
This graph is a **probabilistic roadmap (PRM)**.

**Reuse requires unchanged geometry and checking rules.**

This example includes the endpoints before building; its counts include the query.

Kavraki, Švestka, Latombe & Overmars (1996)

# A graph search selects a route through the roadmap




**Seed 7:** 59 valid samples; 187 accepted edges. Route: **2 edges, 70.05°**.

Shortest in this graph; not a claim about every possible path.

# Expand the cheapest known route first

**Dijkstra's search on nonnegative edge lengths**

1. Start cost is zero; all other tentative costs are infinite.
2. Settle (declare final) the unsettled node with smallest tentative cost.
3. Try each neighbor through that node; keep an improvement.
4. Stop when the goal is settled; follow parent pointers backward.

$$D(v) \leftarrow \min\{D(v),\ D(u) + w(u, v)\}$$

$D$: route length from start. $w$: checked edge length.

**Seed 7:** 7 nodes settled before the goal; route via **(19.20°, 82.74°)**.

# Grow a tree from the current configuration

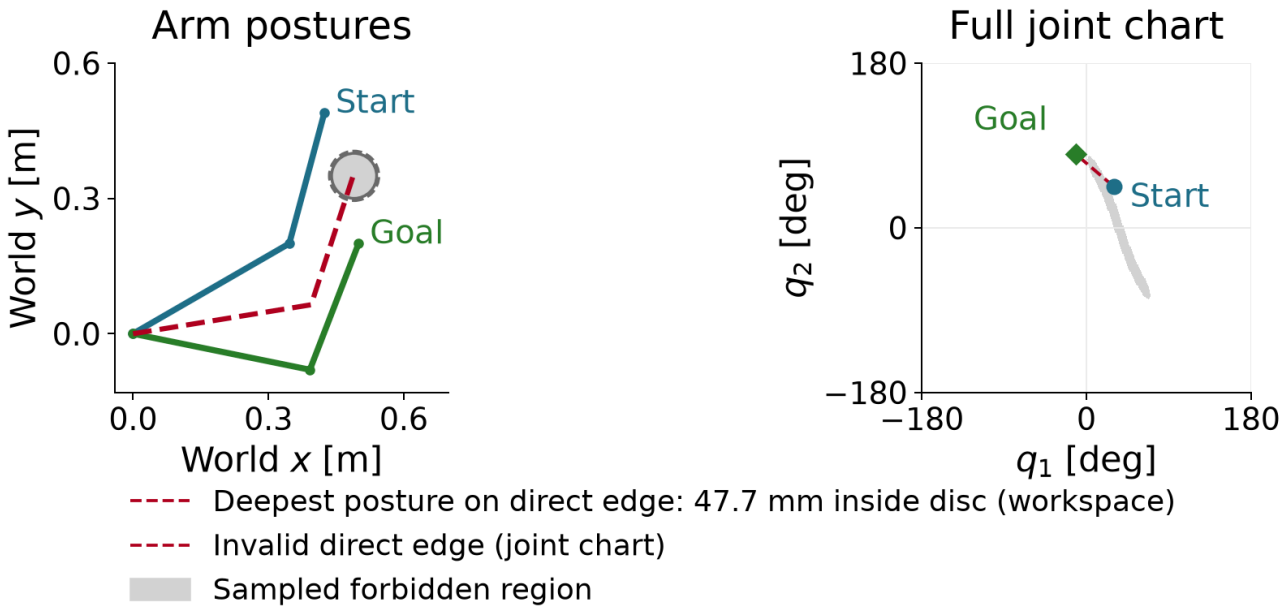
1. Sample $\mathbf{q}_{\text{rand}}$ (the goal with probability 0.1).
2. Find the nearest tree node $\mathbf{q}_{\text{near}}$.
3. Move at most 10° toward the sample to propose $\mathbf{q}_{\text{new}}$.
4. Check the edge; add the new node and parent only if it passes.
5. Try a goal connection when the goal is within 10° of $\mathbf{q}_{\text{new}}$.

This is a **rapidly exploring random tree (RRT)**.

**The sample can be invalid. The added edge must pass.**

LaValle (1998); LaValle & Kuffner (2001)

# Watch the tree decide whether to grow




**Seed 7, paused.**

Zoom changes only the view.

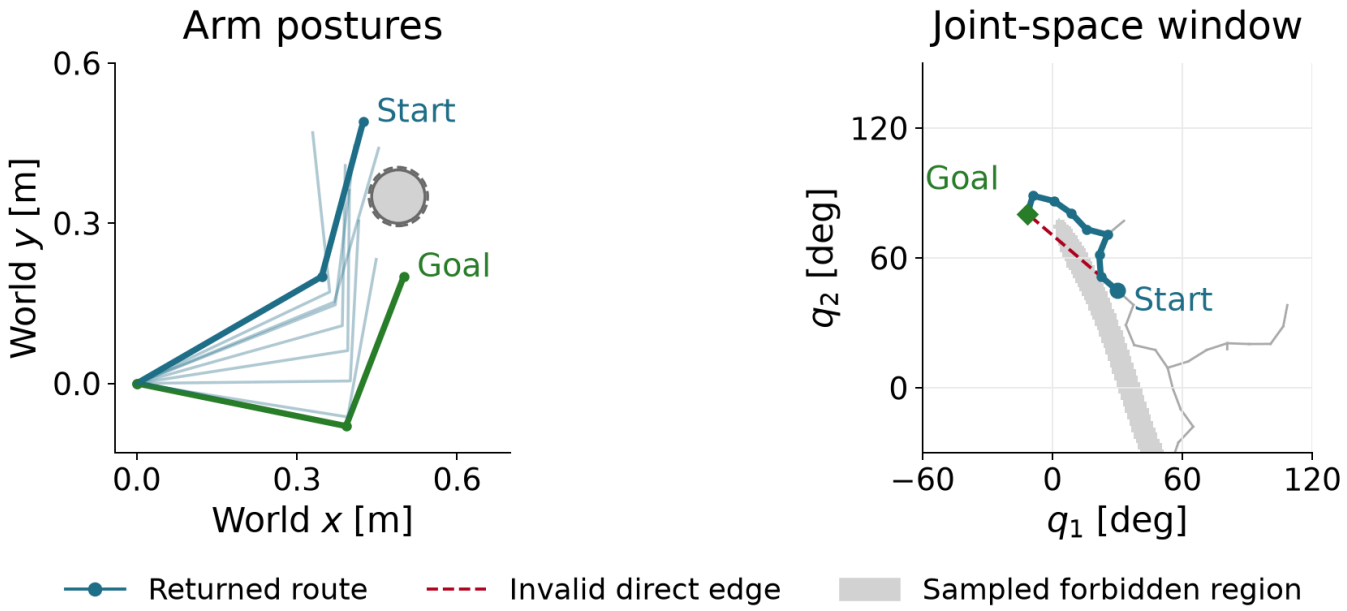
**Predict before each Step:**

Which posture will be tested?

Will the tree grow?

rrt_two_link.py ▶ step through RRT

# Follow the parents to recover a candidate path




**Seed 7**

Goal joins at iteration **42**.

28 nodes; path length **78.71°**.

**Recheck**

74 scheduled postures.

Minimum clearance **10.6 mm**.

**Playback pace is illustrative; execution timing remains to be chosen.**

rrt_two_link.py ▶ **finish the RRT**

# Let two trees try to meet

1. Root one tree at the start and one at the goal.
2. Extend one tree once toward a random sample.
3. If a node was added, extend the other tree toward it until reached or blocked.
4. Finish if the trees meet; otherwise swap roles and continue.

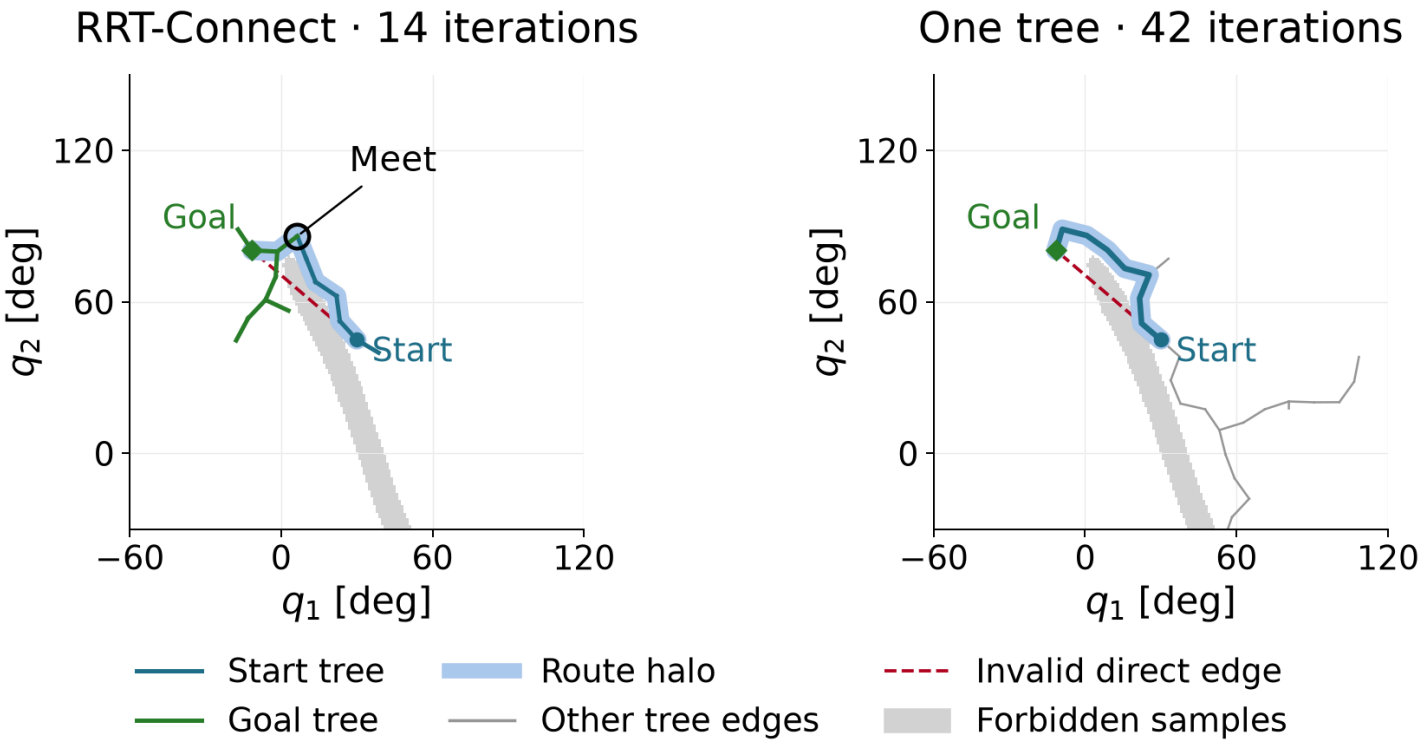
This bidirectional method is **RRT-Connect**.

Each extension keeps the **same edge check and 10° cap**.

Joining reverses the goal chain; these geometric edges can be traversed both ways.

Kuffner & LaValle (2000)

# Fewer iterations need not mean less checking work




| Measure | Seed 7: Connect / RRT | Seeds 1–12: Connect wins | Median: Connect / RRT |
|---|---|---|---|
| Iterations | 14 / 42 | 11 of 12 | 22 / 42.5 |
| Configuration evaluations | 186 / 291 | 4 of 12 | 302 / 287 |

# A search budget is different from a guarantee

**20 iterations, seed 7:** 11 nodes; no connection found.

**42 iterations, same seed:** the goal connects.

A finite failure does not prove infeasibility.

**Probabilistic completeness:** under suitable assumptions, the probability of finding a valid route approaches one as samples increase.

Requires a route with clearance to spare, suitable sampling and connections, and sound validity tests.

**This theorem does not certify our finite checker or five-neighbor roadmap.**

Karaman & Frazzoli (2011) ▶ exhaust a small budget

# Continuing the search can also improve its route

**RRT:** the first accepted parent supplies a route.

**RRT*:** consider nearby parents with lower route cost, then rewire neighbors when a checked connection improves their cost.

- Every replacement edge needs validation.
- Parent changes require descendant cost updates.
- Use a stated radius or neighbor-count rule as samples grow.

**Asymptotic optimality is a limit statement, not a finite-run certificate.**

Karaman & Frazzoli (2011)

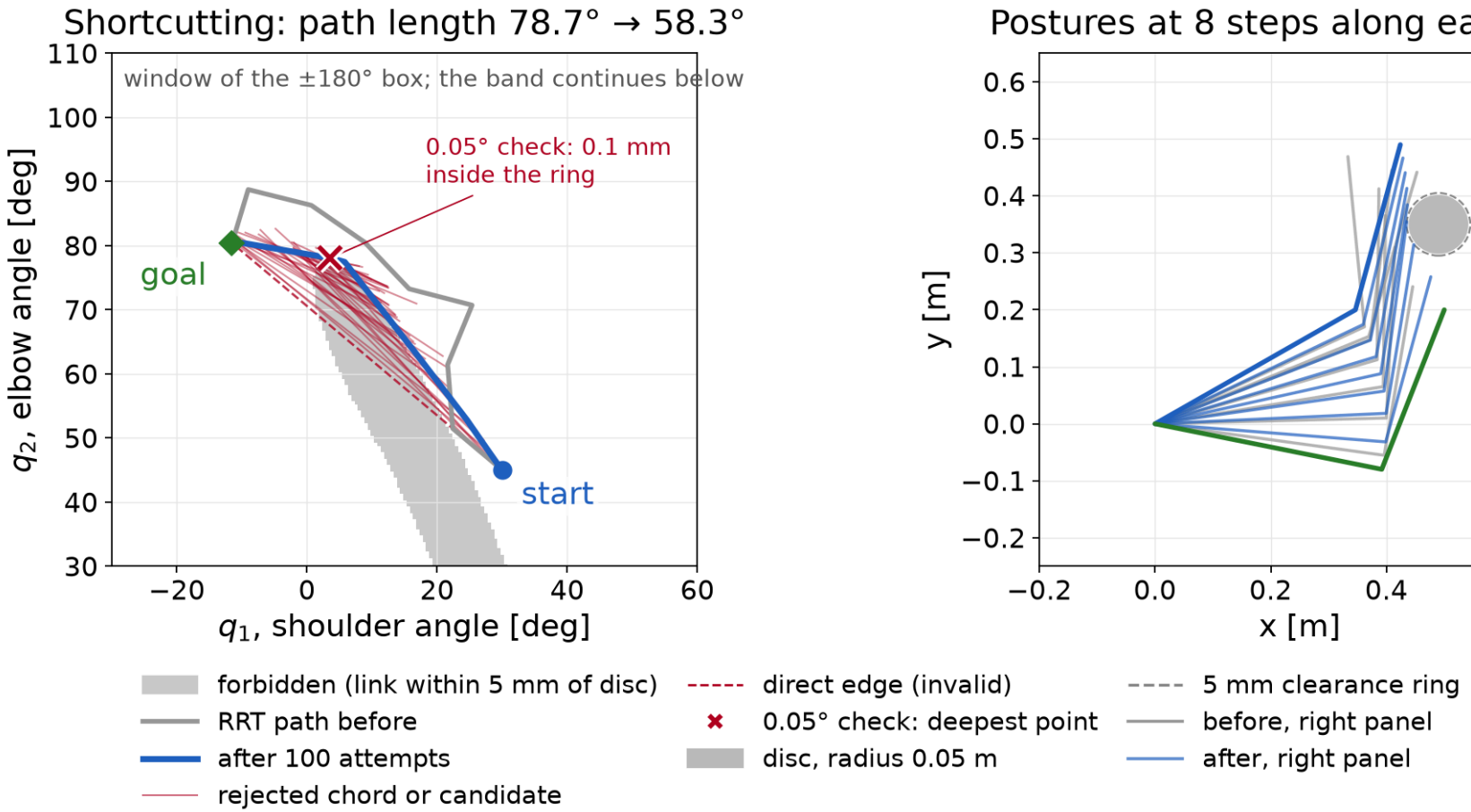
# Try a shorter connection within the returned path

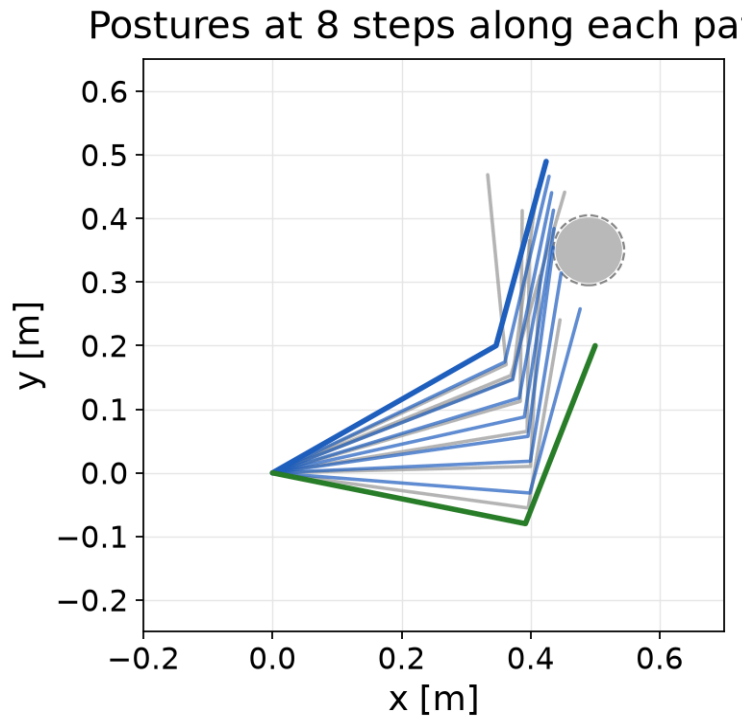
1. Choose two positions along the current path.
2. Connect their configurations by a straight joint-space edge.
3. Check the whole candidate after splitting and replacing edges.
4. Keep a passing candidate; otherwise retain the old path.

This is **shortcutting**.

A shorter geometric route still has corners and no timing.

# The shorter route passes one grid and fails a finer grid




**Length:** 78.71° → 58.34°. **1° check:** minimum clearance 5.1 mm.

**0.05° check:** about 0.1 mm inside the required 5 mm ring.

# Angular spacing must be related to robot motion

Link lengths are $l_1 = 0.4\,\mathrm{m}$, $l_2 = 0.3\,\mathrm{m}$.

For any material point on either link under joint-linear interpolation:

$$\|\Delta \mathbf{p}\| \le (l_1 + l_2)|\Delta q_1| + l_2|\Delta q_2|$$

For 1° changes in both joints:

$$(0.7 + 0.3)\frac{\pi}{180}\ \mathrm{m} \approx 17.45\ \mathrm{mm}$$

From the nearer checked endpoint: **half the bound, at most 8.73 mm**, for any point on either link.

**Sufficient on every subinterval:** both endpoint clearances ≥ **13.73 mm**.

The shortcut's **5.1 mm** does not meet that condition.

# Give optimization the same endpoint and geometry

**Matched program:** fixed positive-elbow goal, both whole links, 5 mm margin.

$$\mathbf{h} = \begin{pmatrix} \mathbf{q}_T - \mathbf{q}_{\text{goal}} \\ (\mathbf{q}_T - \mathbf{q}_{T-1})/\Delta t \end{pmatrix} = \mathbf{0}$$

$$g_{t,\ell} = r_o + d_{\text{req}} - \text{dist}(\mathbf{c}, \text{link}_\ell(\mathbf{q}_t)) \leq 0$$

$t = 1, \ldots, T$, $\ell \in \{1, 2\}$; $T = 20$, $\Delta t = 1/T$, $d_{\text{req}} = 5\,\text{mm}$.

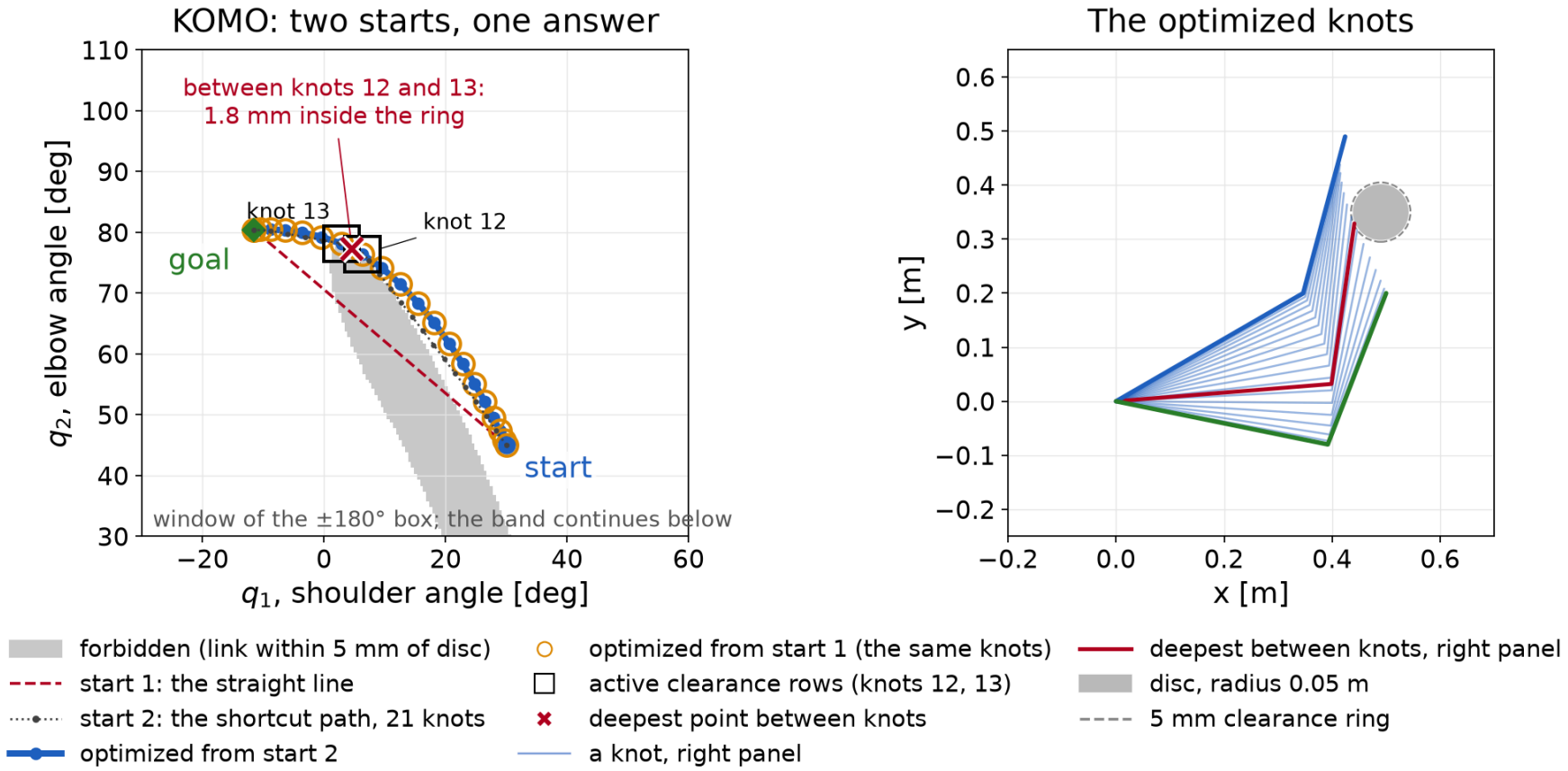
$\text{dist}$: distance from the center to the closest point on link segment $\ell$.

Same acceleration cost and prefix. **Use the sampled route as the initial guess.**

Joint box: checked afterward; no bound rows in this program.

**Resampling the shortcut creates new edges. Recheck them.**

# Both initial guesses reach the same knots here




**NumPy, both starts:** cost 16.1382; five outer iterations to the KKT stop.

**Knot residuals pass; between knots, the 5 mm margin fails by 1.8 mm.**

# Track what changes at each refinement step

| Candidate | Acceleration cost on 21 knots | Whole-link validation |
|---|---|---|
| Original shortcut polyline | — | Passes 1°; fails 0.05° by 0.1 mm |
| Resampled shortcut initialization | $28.78$ | Knots pass; new edges fail by 0.3 mm |
| Optimized, either start | $16.14$ | Knot rows pass (KKT stop); edges fail by 1.8 mm |

**40 intervals:** violation falls to about 0.4 mm; still fails.

All depths are **inside the 5 mm ring**. More knots do not certify clearance.

# Choose timing after checking the route

$$\mathbf{q}(t) = \mathbf{q}_{\text{path}}(s(t))$$

$$\dot{\mathbf{q}} = \mathbf{q}'_{\text{path}}(s)\,\dot{s}, \qquad \ddot{\mathbf{q}} = \mathbf{q}''_{\text{path}}(s)\,\dot{s}^2 + \mathbf{q}'_{\text{path}}(s)\,\ddot{s}$$

$t$: seconds again; $s \in [0, 1]$: path progress; $' = d/ds$, dot $= d/dt$.

**Uniformly double the duration of a smooth trajectory:**

| Quantity | Scale |
|---|---|
| Joint velocity | $1/2$ |
| Joint acceleration | $1/4$ |

A corner requires smoothing or a suitable stop; slowing alone does not remove it.

# Validate the motion that will actually be sent

| Requirement | What must be checked |
|---|---|
| Geometry | Actual robot model, obstacles, clearance and self-collision |
| Connection | The execution interpolant between waypoints |
| Motion limits | Joint positions, speeds and accelerations under its timing |
| Actuation | Required torques or other interface limits, using a dynamics model |

**A modified route or interpolation rule needs renewed geometry checks.**

**A changed schedule needs renewed motion-limit checks.**

# A controller tracks the timed reference

**Planner output:** a validated route and a schedule checked against motion limits.

**Controller input:** reference motion and measured robot state.

**Controller output:** commands through a stated robot interface.

$$\text{reference} \longrightarrow \text{controller} \longrightarrow \text{robot}$$

$$\text{robot} \longrightarrow \text{measured state} \longrightarrow \text{controller}$$

Tracking error and model error still require feedback and monitoring.

# State what the planned motion establishes

- **Optimization:** adjust coupled knots under written costs and constraints.
- **Sampling:** search checked connections for a route.
- **Refinement:** shorten or smooth, then validate the changed motion.
- **Execution handoff:** choose timing, check limits, and track the reference.

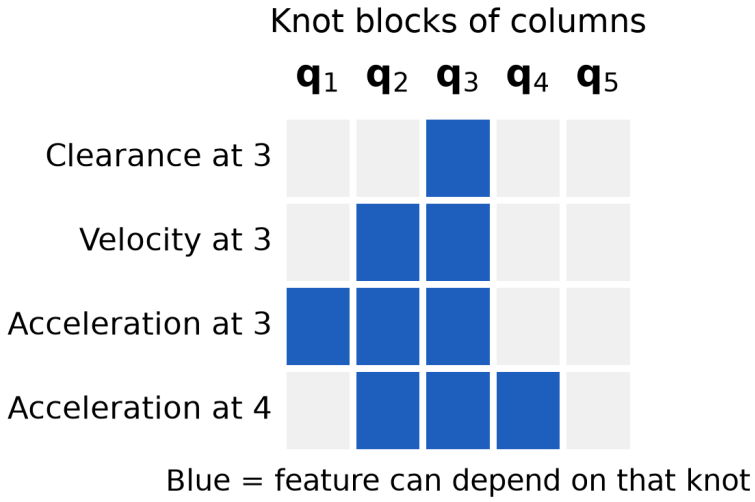
**Next, session 5.1:** learned samplers, feasibility predictions and search heuristics; retain validation.

R 5: quiz 2 (Gauss-Newton, KKT, KOMO, control stack, RRT); A1 due.

# Backup slides

Retained Jacobian bookkeeping, operation counts, solver comparison and API mapping.

# Backup · Locality and operation counts




**Full feature Jacobian:** $84 \times 40$ for 20 two-joint slices.

**The normal matrix is block-banded.**

Estimated solve work: banded $O(T(k+1)^2 n^3)$; dense $O((nT)^3)$.

# Backup · Two Jacobians, both sparse

| Rows | Full feature Jacobian | Inner residual Jacobian at solution |
|---|---|---|
| Smoothness | $40$ | $40$, times $\sqrt{2}$ |
| Endpoint and rest | $4$ | $4$, times $\sqrt{2\mu}$ |
| Whole-link clearance | $40$ | $2$ active rows, times $\sqrt{2\mu}$ |
| Size | $84 \times 40$ | $46 \times 40$ |

**Only link 2 at knots 12 and 13 has a positive multiplier.**

The active set and multiplier shifts can change the inner residual system.

# Backup · Two starts in the matched program

| | Straight-line start | Resampled shortcut start |
|---|---|---|
| Initial knot clearance | Fails by $52.7$ mm beyond the ring | Passes; nearest is $0.48$ mm outside ring |
| NumPy final cost | $16.1382$ | $16.1382$ |
| NumPy active rows | Link 2, knots 12–13 | Same |
| NumPy stop | KKT thresholds, 5 outer iterations | Same |
| Fixed-$\mu$ RAI result | Same knots within $3 \times 10^{-9}$ rad | Same knots within $3 \times 10^{-9}$ rad |
| Output edge check | Fails 5 mm rule between knots | Same |

**RAI's default penalty schedule reaches its evaluation budget with residual violations.**

staged_two_link.py · stage4_komo_rai.py · sampling-demo/evidence/rai-stage4/run.txt

# Backup · The shape-based program in RAI

```
komo = ry.KOMO(C, phases=1, slicesPerPhase=20,
               kOrder=2, enableCollisions=False)
komo.addControlObjective([], 2, 1.)
komo.addObjective([1.], ry.FS.qItself, [], ry.OT.eq, [1.], q_goal)
komo.addObjective([1.], ry.FS.qItself, [], ry.OT.eq, [1.], order=1)
for link in ("link1", "link2"):
    komo.addObjective([], ry.FS.negDistance,
                      [link, "disc"], ry.OT.ineq, [1.])
```

Capsule links and an adjusted disc sphere reproduce the segment-distance rows.

**Fixed-$\mu$ solver:** matched result; **default schedule:** evaluation budget reached.

Joint bounds are checked afterward; this program has no bound rows.

modules/motion-planning/stage4_komo_rai.py · rai-stage4/run.txt

# Backup · One free knot recovers a posture cost

$$T = 1, \qquad \Delta t = 1, \qquad \mathbf{q}_{-1} = \mathbf{q}_0$$

$$\mathbf{r}_1 = \mathbf{q}_1 - \mathbf{q}_0, \qquad f = \|\mathbf{q}_1 - \mathbf{q}_0\|^2$$

**Deck 05's convention:**

$$\frac{1}{2}\|\mathbf{q} - \mathbf{q}_{\text{ref}}\|^2_{\mathbf{D}}, \qquad \mathbf{q}_{\text{ref}} = \mathbf{q}_0, \quad \mathbf{D} = 2\mathbf{I}$$

Omit terminal rest; with one interval it would force the only step to zero.